## LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

**Section:** 10. List

## Lecture: 110. Nested List

**Section 1: Lecture Summary**

This lecture introduces the concept of **nested lists** in Python, which are lists that contain other lists as elements. It explains the structure of nested lists through several examples, demonstrating how lists can be embedded within lists at multiple levels (multi-level nesting). The lecture covers the creation of nested lists, accessing elements within them using multiple indices, and their practical use cases such as representing matrices (2D arrays) and tabular data. It highlights the flexibility of Python lists in storing heterogeneous data types in nested formats. Finally, it discusses iterating through nested lists using nested loops and previews future lectures applying nested lists in projects.

---

**Section 2: Key Concepts and Explanations**

- **Nested List Definition**

  - A nested list is a list that contains other lists as its elements.

  - Nested lists can represent complex data structures like matrices or tables.

  - Nesting can be multi-level, e.g., a list inside a list, which is inside another list.

- **Creating Nested Lists**

  - You can add lists as elements inside another list.

  - Example: `L2 = [[1, 2], 3, 4, 5]` means the first element is a list, followed by integers.

  - Nested lists can have varying depths, for example:

    - `L4 = [[[1, 2]], [5, 6]]` shows a three-level nesting, where `[1, 2]` is inside another list which is the first element of `L4`.

- **Accessing Elements in Nested Lists**

  - Use multiple indices to access elements inside nested levels.

  - Syntax: `list[index1][index2][index3]...` depending on the nesting depth.

  - Example:

- `L1` accesses the first element of the outer list.

- `L1[1]` accesses the second element of the first sublist.

- For an element in a matrix represented as a nested list, use row and column indices. For example, `matrix[2][3]` accesses the element at row 2 and column 3.

- **Use Cases**

- Matrix Representation: Use nested lists where each sublist represents a row.

- Tabular Data: Nested lists can represent tables where each sublist corresponds to a row including heterogeneous data types such as strings and integers (e.g., item name, type, and price).

- **Iteration over Nested Lists**

- Use nested `for` loops:

- Outer loop iterates over rows (outer list).

- Inner loop iterates over elements within each sublist.

- This is essential for traversing or processing matrices.

- **Lists Support Heterogeneous Data**

- Lists in Python can hold mixed data types.

- Important when representing data like tables with string and numeric columns.

---

**Section 3: Example Code and Use Cases**

### Example 1: Simple nested list with one sublist and individual elements

```
L2 = [[1, 2], 3, 4, 5]
print(L2)
```

Output: `[[1, 2], 3, 4, 5]`

Explanation: The first element is a list `[1, 2]`, others are integers.

### Example 2: Nested list with multiple sublists

```
L3 = [[1, 2], [3, 4], [5, 6]]
print(L3)
```

Output: `[[1, 2], [3, 4], [5, 6]]`

Explanation: Outer list has three elements, each is a sublist of two integers.

### Access elements in nested lists

```
print(L3[0])    # Output: [1, 2] - first sublist
print(L3[0][1]) # Output: 2 - second element of the first sublist
print(L3[2][0]) # Output: 5 - first element of the third sublist
```

### Nested list with 3-level nesting

```
L4 = [[[1, 2]], [5, 6]]
print(L4)
print(L4[0])       # Output: [[1, 2]]
print(L4[0][0])    # Output: [1, 2]
print(L4[0][0][1]) # Output: 2
```

### Example 3: Representing a 4x4 matrix using nested lists

```
matrix = [
    [1, 2, 3, 4],
    [3, 5, 7, 9],
    [2, 4, 6, 8],
    [4, 3, 2, 1]
]
print(matrix)
print(matrix[2][3])  # Output: 8 (element at third row, fourth column)
```

### Iterating through a matrix (nested list)

```
for i in range(len(matrix)):        # Rows
    for j in range(len(matrix[i])): # Columns
        print(matrix[i][j], end=' ')
    print()
```

Output:

```
1 2 3 4
3 5 7 9
2 4 6 8
4 3 2 1
```

### Example 4: Representing tabular data (items with name, type, price)

```
items = [
    ['Soap', 'Bathroom', 125],
    ['CloseUp', 'Toothpaste', 200],
    ['Sunsilk', 'Shampoo', 200],
    ['Dairy Milk', 'Chocolate', 50]
]
print(items)
```

```
print(items[1])       # ['CloseUp', 'Toothpaste', 200]
print(items[1][2])    # 200
```

Explanation: Each row represents an item with heterogeneous elements — strings and integers.

---

**Section 4: Key Takeaways**

- Nested lists allow you to store lists inside other lists, enabling representation of complex data structures.

- Accessing elements in nested lists requires multiple indices corresponding to each nesting level.

- Python lists can contain mixed data types, which is useful for tabular data representation.

- Nested lists are ideal for representing matrices, tables, or any structured multi-dimensional data in Python.

- Iterating through nested lists uses nested loops to access all elements efficiently.

- Multi-level nesting is supported and can be used to build very complex data representations.

- Practicing nested lists prepares you for using them in real projects involving structured data.